

基于 RK3588 芯片的视觉雷达融合防撞与微秒级精准 追溯黑匣子系统

摘要

本项目面向无围栏工业人机协作场景，研发了一款基于 RK3588 异构算力平台的一体化边缘计算智能终端。终端采用 3D 打印一体化结构设计，集成左右双路 1080p 摄像头、360°单线激光雷达、NVMe SSD、高精度六轴 IMU 等硬件，构建了视觉、雷达、惯性与存储一体化的多元感知平台。

针对工业设备运行过程中的机械振动，系统基于 ICM45686 IMU 构建电子图像防抖（EIS）链路，通过 IMU 坐标标定、双相机外参映射、姿态积分与 RGA 硬件裁剪偏移补偿，提升振动环境下视频输入稳定性。在安全感知方面，系统集成 YOLOv8 NPU 推理与 LiDAR-Camera 外参融合算法，结合 Alpha-Beta 多目标跟踪器，实现人员检测、跨视角连续追踪、风险区域识别与安全告警。数据记录方面，系统在内核块设备层实现面向 NVMe SSD 的环形黑匣子机制，通过 bio 扇区重映射完成裸块循环写入，降低传统文件系统元数据 I/O 抖动与写放大。系统基于 MPP 硬件编码实现低延迟 RTSP 推流和 1080p/720p 本地 MP4 录像，推流与录像双编码器可独立并行运行；同时提供 Qt5 触控 HMI 与 Web 远程控制界面，通过 REST API 与 WebSocket 实现双向互控和状态同步。进一步地，系统在 RK3588 端部署 DeepSeek-R1-Distill-Qwen-1.5B 量化大模型，基于 RKLLM Runtime 构建 AI 运行状态分析模块，采集 CPU、双相机、LiDAR、IMU 与融合跟踪等运行状态，自动生成系统健康度评估、异常风险分析和运维优化建议。全链路采用 DMA-BUF 零拷贝架构，充分利用 RK3588 的 RGA、NPU、MPP 等异构硬件能力，形成集感知、推理、跟踪、记录和交互于一体的工业安全边缘智能终端。

第一部分 作品概述

1.1 功能与特性

本终端面向无围栏工业人机协作场景，提供以下核心功能与特性：AI 智能日志分析——基于 DeepSeek-R1-Distill-Qwen-1.5B 板端推理，对 CPU、双相机、LiDAR、IMU 和融合跟踪状态生成健康评估、异常分析与运维建议；实时安全感知——通过 YOLOv8 NPU 推理与 LiDAR 点云融合，实现 360°无死角的人员检测与定位，百毫秒级告警响应；电子图像防抖——基于 ICM45686 六轴 IMU 与 RK3588 RGA 硬件加速实现 IMU 电子图像防抖，通过 IMU 姿态解算、相机外参映射与 RGA 裁剪偏移补偿，提升振动环境下视觉输入的稳定性；跨视角连续追踪——Alpha-Beta 多目标跟踪器支持跨视角的人员身份保持，跨视角跟踪成功率高达 95 %；视频推流与录像——MPP 硬件 H.264 编码，RTSP 推流延迟 164ms，支持 OSD 叠加（检测框+雷达点云），本地 MP4 录像支持 1080p/720p 双分辨率可切换，推流与录像双编码器独立可同时运行互不干扰；环形黑匣子记录——基于 dm-ringbox 内核驱动，NVMe 裸块环形写入速度达 369.5MB/s，写放大系数 WAF=1.0，支持事件触发式数据回溯；双终端管控——嵌入式 Qt5 触控 HMI + Web 远程控制，Qt 内嵌 HTTP/WebSocket 服务器，通过 REST API 与 WebSocket 实时推送实现双向互控与状态实时同步，系统功能特性如图 1 所示。



图 1 系统功能特性概述图

1.2 应用领域

1. 工业制造安全：在汽车焊接、冲压、总装等柔性制造产线中，本终端可部署于协作机器人工作站、AGV运行通道等关键位置。当检测到人员进入机器人操作半径或AGV行驶路径时，系统在百毫秒内触发设备减速或急停。
2. 仓储物流监控：在智能仓储和物流分拣中心，本终端可同时监控多台移动机器人与人员的动态位置关系，构建动态电子安全围栏。相比传统安全光幕，覆盖范围大、安装灵活、可随布局调整重新配置。
3. 人机协作优化：系统记录的连续多传感器数据（视频、雷达点云）可用于事后分析人机交互模式、识别危险行为高频区域、量化安全裕度，为产线布局优化和操作规程改进提供数据支撑。
4. 事故调查与合规：黑匣子机制确保事故前关键时段（可配置，默认5秒）的完整数据被不可篡改地保存，满足《安全生产法》对安全数据记录的要求。详细应用展示如图2所示。



图2 应用领域示意图（工业制造、仓储物流、人机协作三大场景）

1.3 主要技术特点

1. 异构融合感知：融合 YOLOv8 视觉检测与单线 LiDAR 点云，结合外参变换、投影关联、DBSCAN 聚类、bbox 认领和 Alpha-Beta 跟踪，实现目标级稳定感知。
2. IMU 电子防抖：基于 ICM45686 角速度积分估计姿态变化，并将补偿量注入

RGA 裁剪缩放流程，实现低延迟图像防抖。

3. 全链路零拷贝：V4L2 采集、RGA 处理、NPU 推理和 MPP 编码之间通过 DMA-BUF 传递图像数据，减少 CPU 像素搬运，提升 RK3588 异构算力利用率。
4. 可靠录像与黑匣子存储：采用 RTSP 推流与本地 MP4 录像双编码架构，并基于 Device Mapper 实现 dm-ringbox 裸块环形写入，降低 I/O 抖动与写放大。
5. 双终端互控：Qt 触控端与 Web 远控端共用后端，通过 HTTP/WebSocket 与跨线程同步调用，实现状态实时同步和双向控制。
6. AI 运行状态分析：基于 DeepSeek-R1-Distill-Qwen-1.5B 量化模型与 RKLLM Runtime 构建板端 AI 分析模块，自动生成健康评估、异常风险与运维建议。

1.4 主要性能指标

性能指标如下表 1 所示。

表 1 性能指标

指标类别	技术参数	指标数值
处理器与 AI 算力	RK3588 (4×A76+4×A55), NPU INT8	6 TOPS
左视摄像头	MIPI-CSI ISP, OV13855	1920×1080@30fps, NV12
右视摄像头	USB UVC	1920×1080@30fps, YUYV
激光雷达	镭神 N10Plus 单线 TOF	10Hz, 540 点/圈, 0.15-50m
IMU	TDK ICM45686 6 轴 MEMS	SPI 100Hz, ±2G/±250dps
EIS 防抖	ICM45686 IMU + RGA 硬件 偏移补偿	IMU 姿态驱动，支持双相机外参配置，RGA 裁剪缩放同步补偿，处理延迟 <5ms
目标检测	YOLOv8n INT8 NPU	27ms 帧, mAP@0.5=0.453
多目标跟踪	Alpha-Beta 融合跟踪	跨视角跟踪成功率（单人） 95%
视频推流	MPP H.264 RTSP	端到端延迟 164ms (有线)
存储性能	dm-ringbox NVMe	369.5MB/s, WAF=1.0

功耗与尺寸	DC 12V/5A, 3D 打印壳体	<30, 约 250×180×120mm
AI 运行状态分析	DeepSeek-R1-Distill-Qwen-1.5B W8A8 RKLLM + AIReportWorker	每秒更新系统状态快照；200ms 请求轮询；推理超时 60s；输出系统健康度、异常风险点与优化建议

1.5 主要创新点

- 采用 3D 打印一体化结构，将双摄像头、LiDAR、IMU、RK3588 与 NVMe SSD 集成于单一壳体，提升部署便利性。
- 基于 IMU 姿态估计与 RGA 裁剪缩放补偿，实现低延迟电子图像防抖。
- 通过 LiDAR-Camera 投影融合与 Alpha-Beta 跟踪，提升跨视角目标保持能力。
- 设计 dm-ringbox 内核黑匣子，利用 bio 重映射实现低写放大环形写入。
- 构建 DMA-BUF 零拷贝链路，减少 V4L2、RGA、NPU、MPP 间 CPU 搬运，有效降低 CPU 利用率。
- 部署 DeepSeek 量化模型，实现运行状态分析、异常识别与运维建议生成。

1.6 设计流程

项目按照“需求分析—系统设计—模块开发—集成调试—测试验证”的 V-Model 流程推进。需求阶段面向无围栏工业人机协作场景，明确人员检测、风险告警、跨视角追踪、振动防抖、录像存储和远程管控需求，并参考工业安全标准确定设计目标。设计阶段完成 RK3588 平台、双摄像头、LiDAR、IMU、NVMe SSD 及 3D 打印一体化结构方案。开发阶段依次实现驱动、感知融合、NPU 推理、EIS 防抖、推流录像、Qt/Web 控制和 DeepSeek AI 运行状态分析模块。集成与测试阶段完成板端部署、软硬件联调、录像回放、延迟统计、AI 状态分析报告生成和现场运行验证。如下图 3 所示。

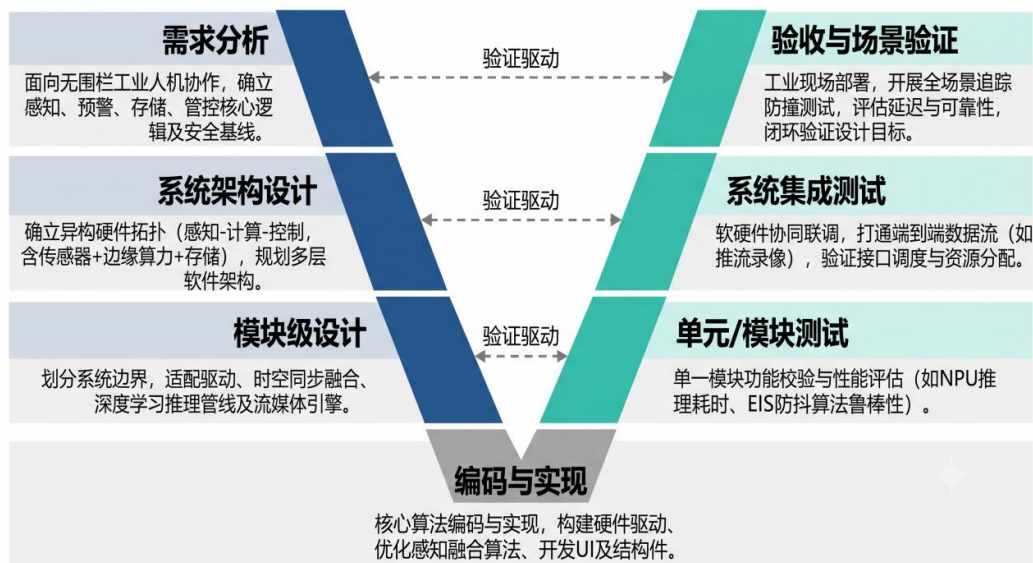


图 3 项目设计流程图（V-Model）

第二部分 系统组成及功能说明

2.1 整体介绍

RK3588-Omni-Sentinel 系统采用七层架构设计，自底向上为：硬件层（RK3588 SoC+双路摄像头+Lidar+IMU+NVMe SSD+DSI 触屏）→ 驱动层（dm-ringbox.ko、icm45686_spi.ko/inv_imu_driver.ko、OV13855 DTS）→ 基础设施层（DMA 内存池、3rdparty RGA/MPP/FFmpeg/RKNN 库、NVMeData Manager）→ 感知层（SentinelVisioner 视觉采集、SentinelLslidarer 雷达驱动、SentinelYoloInfer NPU 推理、icm45686-eis-app IMU 防抖）→ 融合层（LidarCameraFusion Lidar-Camera 融合跟踪）→ 流媒体层（SentinelStreamer RTSP 推流+MP4 录像+OSD 叠加）→ 交互与 AI 运维层（SentinelQT 嵌入式触控 HMI、WebControl Web 远程控制、DeepSeek AI 运行状态分析模块）。各层之间通过明确的公共头文件、运行时 API 和回调接口解耦，支持模块的独立编译、独立测试和渐进式部署。如下图 4 所示，七层架构，图中标注出了各子模块及模块间数据流关系。

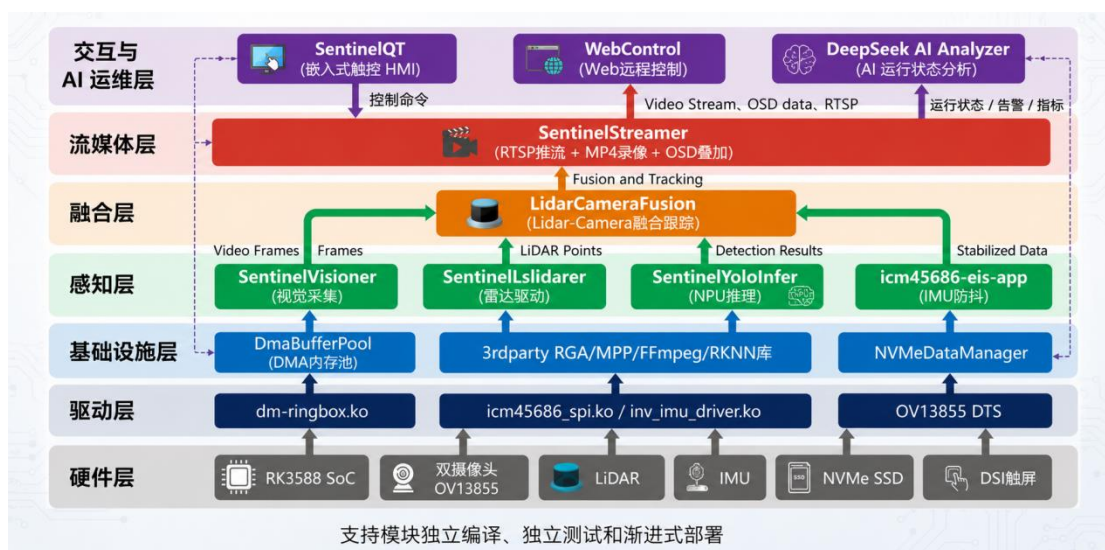


图 4 系统总体架构框图

2.2 硬件系统介绍

2.2.1 硬件整体介绍

系统硬件以 Rockchip RK3588 为核心计算平台（4×Cortex-A76@2.4GHz + 4×Cortex-A55@1.8GHz，内置 6 TOPS INT8 NPU、Mali-G610 GPU 和 RGA2D 硬件加速引擎），外围集成以下传感器与执行模块：左视 OV13855 MIPI-CSI 摄像头（13MP，4-lane，1080p@30fps，NV12 输出，经 RK3588 ISP 处理）；右视 USB UVC 摄像头（1MP，1080p@30fps，支持 YUYV/MJPEG 格式自适应）；镭神 N10Plus 单线 TOF 激光雷达（UART 460800bps，10Hz，0.67°角分辨率，50m 最大量程）；TDK ICM45686 六轴 MEMS IMU（SPI MODE3，1MHZ）；256GB NVMe SSD（PCIe3.0×4，M.2 2280）；7 寸 DSI 电容触控屏（1024×600）；DC 12V/5A 供电，DC-DC 转换提供 5V/3.3V/1.8V 电压轨。所有传感器通过标准数字接口与 RK3588 连接，无模拟信号链路，系统总功耗<30W。如下图 5 所示。

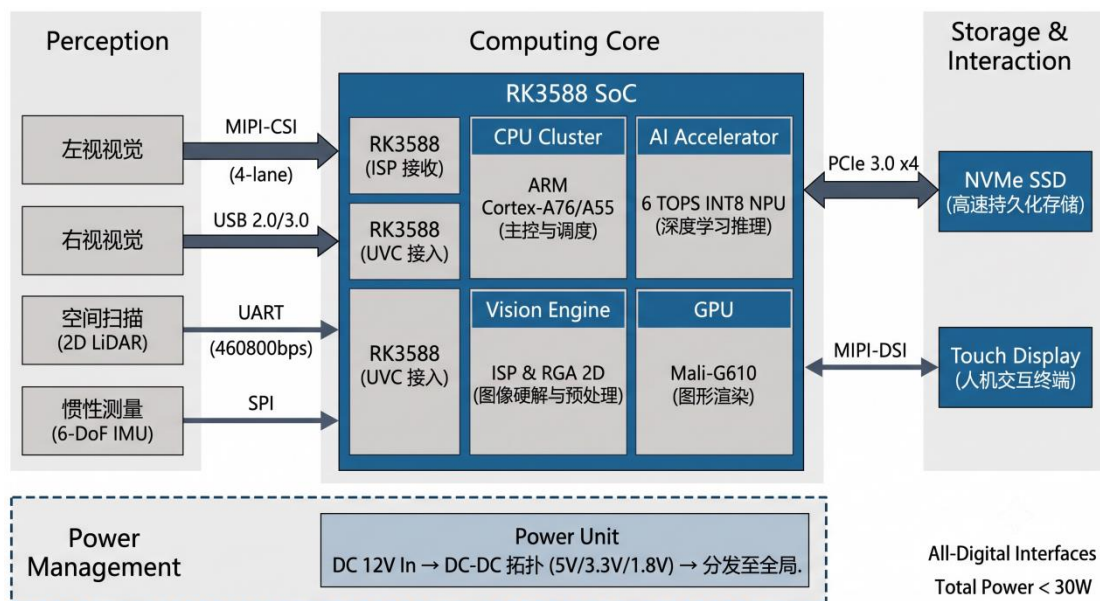


图 5 硬件系统整体框图

2.2.2 机械设计介绍

终端采用 SolidWorks 进行参数化建模。整体结构由四个主要部件组成，通过 FDM 3D 打印（PLA+材料，层厚 0.2mm，填充率 20%）制造验证。整体外形尺寸约 250×180×120mm。装配体文件为装配体.SLDASM（743KB）。

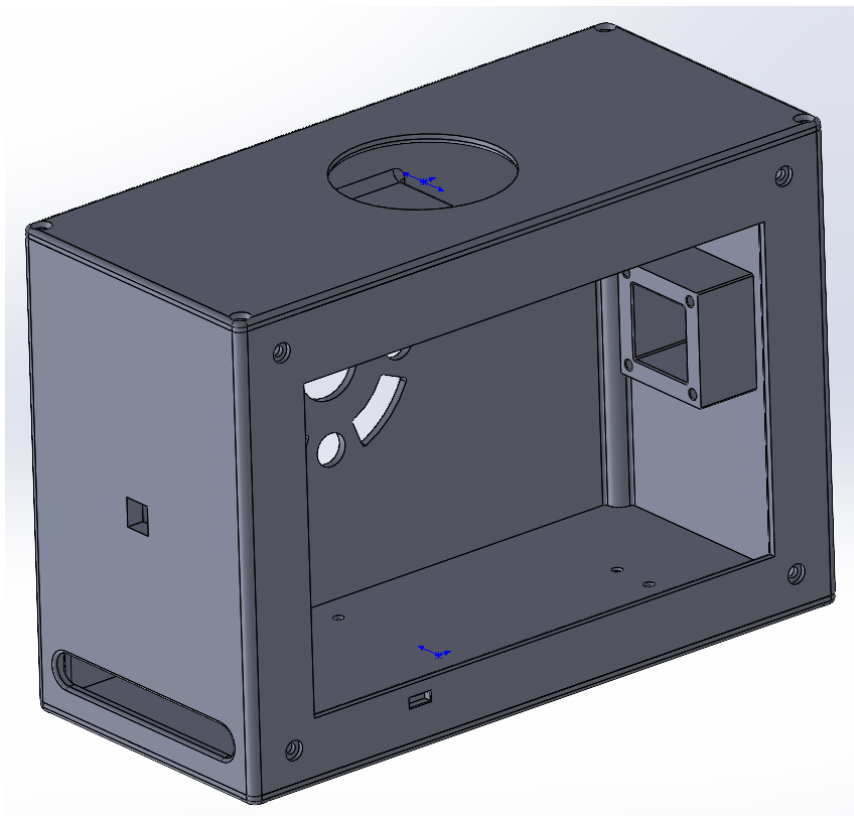


图 6 3D 打印整体装配图

底板（底板.SLDPRT）：承载RK3588核心板和TDK ICM45686六轴MEMS IMU。

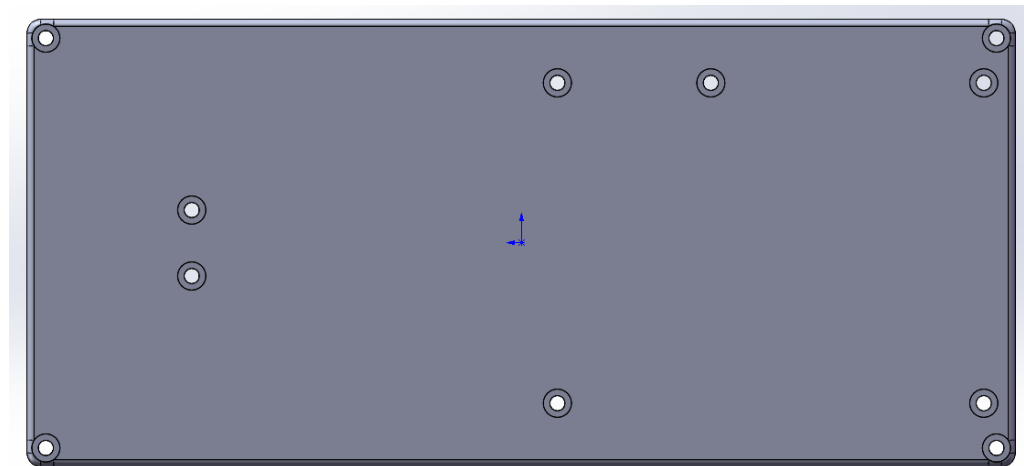


图 7 底板 CAD 设计图

机体（机体.SLDPRT）：主体结构件，集成前后摄像头安装座、7 寸 DSI 屏幕安装框架（安装孔距 180×120mm）、LED 指示灯孔、换气格栅，屏幕有效显示区域 163.8×97mm，背部换气格栅提高散热效率。

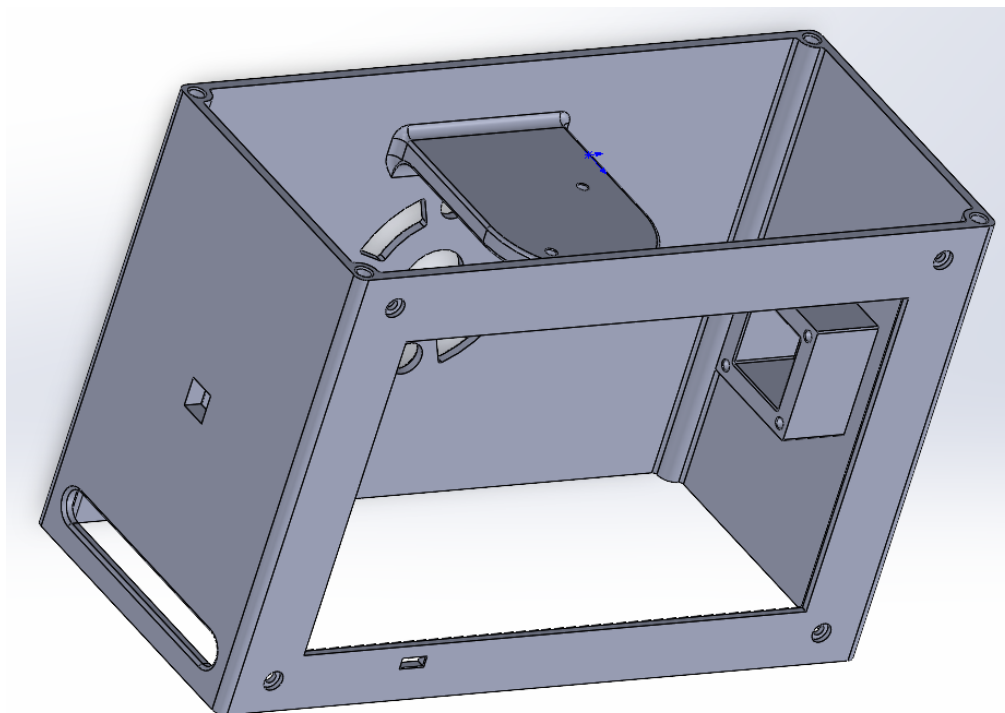


图 8 机体 CAD 设计图

顶盖（顶盖.SLDPRT）：集成LiDAR顶部安装平台、四角安装孔位。

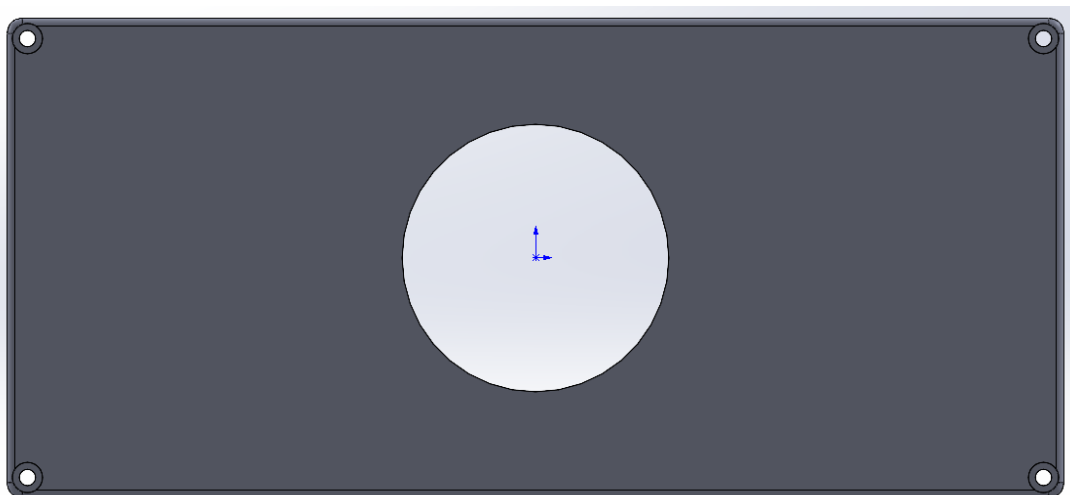


图 9 顶盖 CAD 设计图

摄像头固定件（OV13855 固定件.SLDPRT）：OV13855 专用支架，尺寸适配 8.5×8.5×5mm 摄像头模块。

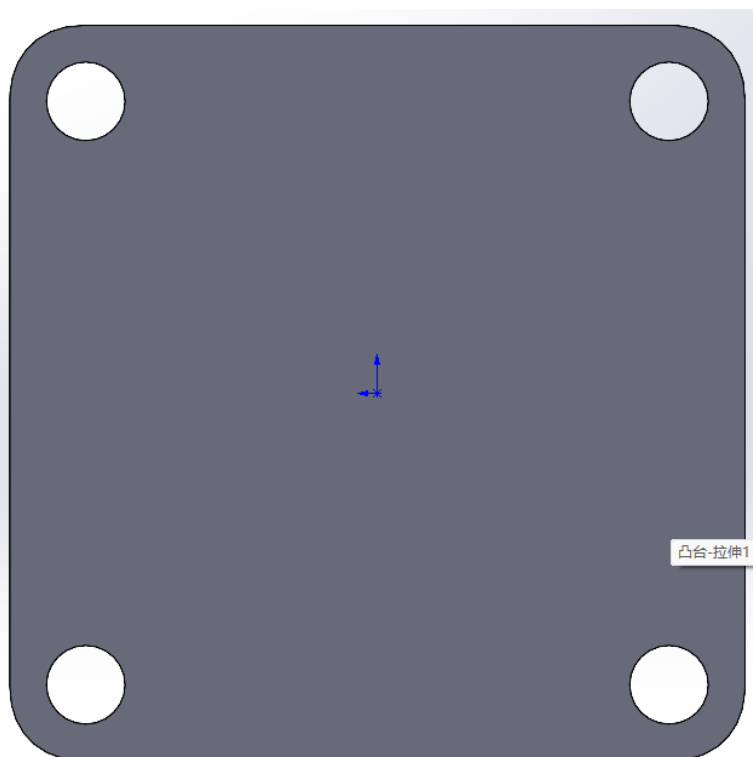


图 10 摄像头固定件 CAD 设计图

2.2.3 电路各模块介绍

系统电路以 RK3588 核心板为中心，各传感器通过标准接口连接。RK3588 通过 MIPI-CSI（4-lane，1.5Gbps/lane）连接 OV13855 左摄像头，通过 USB 接右摄

像头,通过USB转串口连接N10Plus雷达,通过SPI4(MODE3,CPOL=1/CPHA=1,1MHz时钟,CS0片选)连接ICM45686 IMU,通过PCIe3.0×4(8GT/s per lane)连接NVMe SSD,通过DSI(4-lane)连接触控屏幕(触摸I2C地址0x5D)。电源部分采用DC 12V输入→SY8303 DC-DC降压至5V/3A(USB供电)→AMS1117-3.3 LDO提供3.3V/1A(LiDAR/IMU供电)→AMS1117-1.8 LDO提供1.8V/0.5A(摄像头IO供电)。软件侧通过设备树将ICM45686声明为SPI4总线下的icm45686@0节点,compatible设置为"invensense,icm45686",由icm45686_spi.ko在probe阶段完成SPI参数配置、WHO_AM_I校验和/dev/icm45686字符设备创建。

2.3 软件系统介绍

2.3.1 软件整体介绍

本系统提供两套用户界面:嵌入式Qt5触控HMI(SentinelQT,本地操作)和Web远程控制单页应用(WebControl,远端集控),两套界面共享同一后端组件软件栈。SentinelQT基于Qt5 Widgets框架,以eglfs模式直接渲染到DRM/KMS,无Window Manager依赖,实现全屏无边框的嵌入式HMI。且SentinelQT进程内部集成DeepSeek AI,由DeepSeekInference和AIRreportWorker两个核心类组成,实现日志的实时分析。

主界面采用QStackedWidget管理4个功能页面(主控/视频管理/融合管理/数据回溯),通过多线程Worker模式(PreviewWorker×2、FusionWorker、NvmeWorker)将耗时操作从GUI主线程分离。WebControl基于cpp-httpplib实现嵌入式HTTP服务器(27+ REST API路由),通过WebSocket实时推送设备状态、事件和跟踪结果,前端SPA支持桌面端浏览器。两套界面之间的互控通过以下机制实现:Web→Qt方向,HTTP请求到达WebServer工作线程后,通过QMetaObject::invokeMethod(Qt::BlockingQueuedConnection)跨线程同步派发至Qt GUI主线程执行,确保Widget操作线程安全并返回执行结果;Qt→Web方向,Qt主线程通过push_status/push_event/push_tracking三个非阻塞接口将状态JSON推入消息队列(std::queue+mutex),WebServer内部50ms定时器排空队列并通过WebSocket广播至所有连接客户端。该双向通道确保触控操作与远程浏览器操作的状态实时同步、无缝切换。系统运行时配置通过config.ini(QSettings格式,

8 个配置节) 集中管理, 支持在线参数调整和持久化。

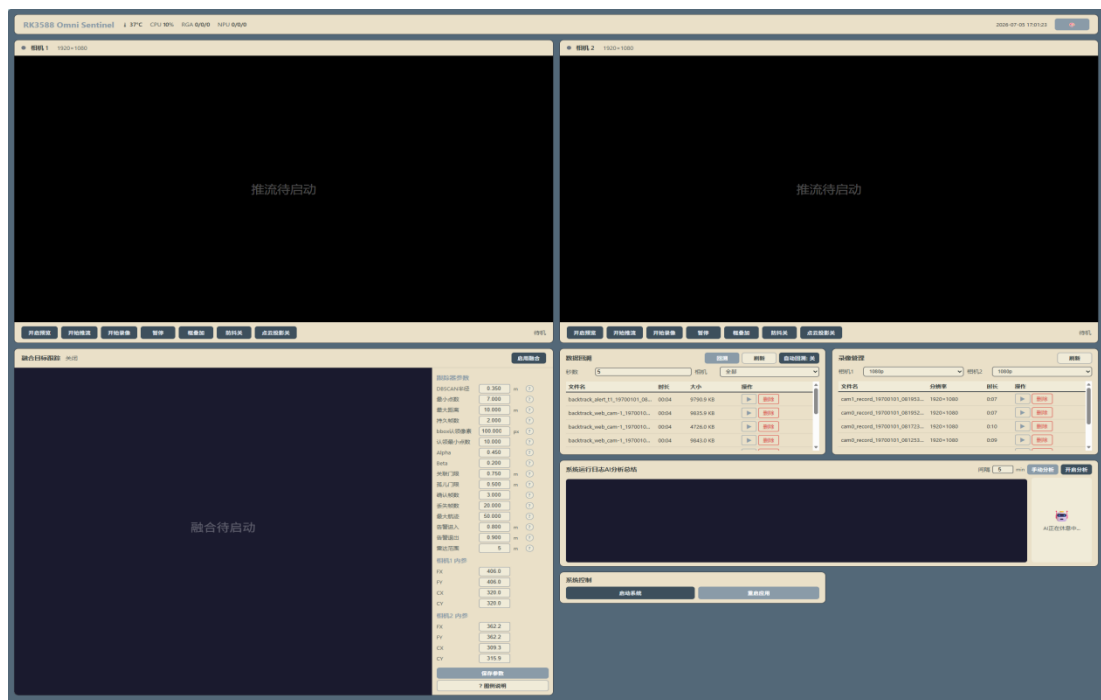


图 11 Web 远程控制前端界面截图

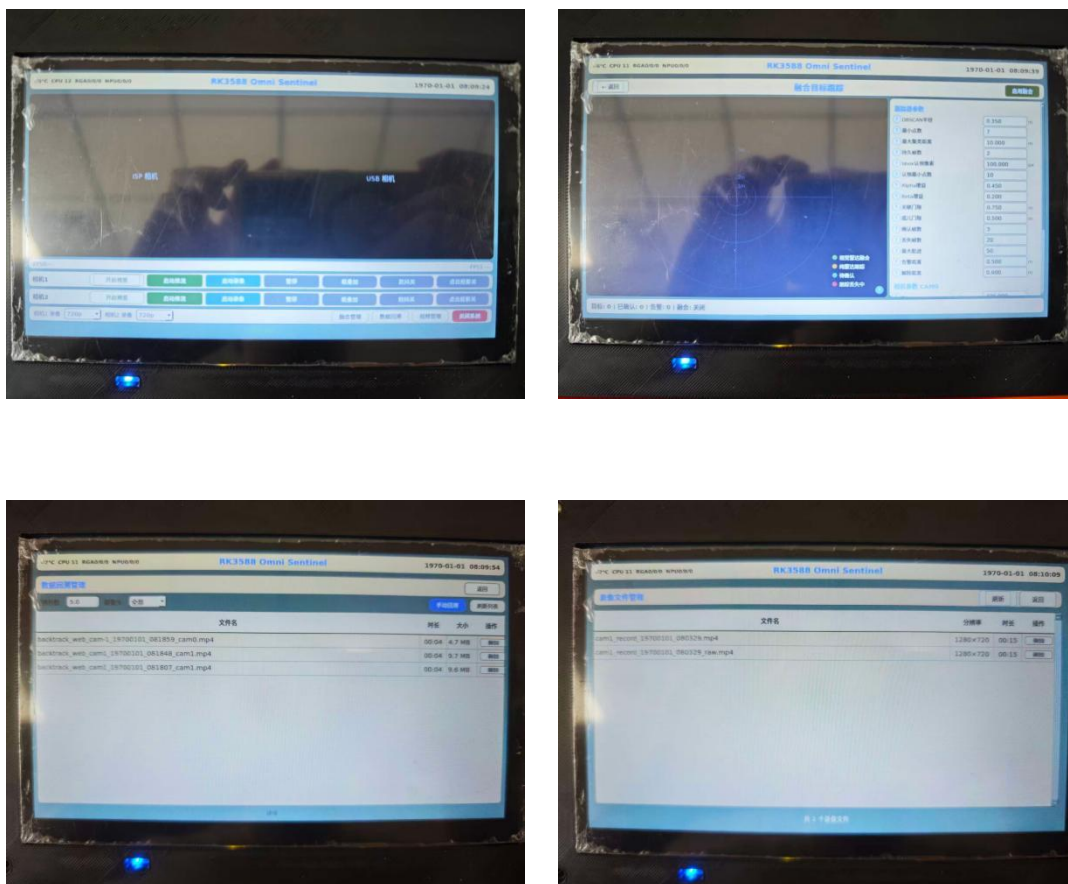


图 12 SentinelQT 嵌入式触控 HMI 主界面截图 (4 页面展示)

2.3.2 软件各模块介绍

系统软件由用户态 C++14 模块、Linux 内核驱动模块及 1 个 AI 运行状态分析扩展组件构成，主要用户态模块编译为静态库，内核驱动以 .ko 模块形式加载，通过 CMake/Makefile 交叉编译至 ARM64 目标平台。以下从顶层到底层逐级说明各模块的架构、流程和输入输出。

1.SentinelQT（应用集成与 AI 运维层）：全部子系统的集成枢纽。输入包括用户触控事件、Web HTTP 请求、config.ini 配置文件以及各模块运行状态；输出包括控制指令（相机启停、推流/录像控制、OSD/EIS 开关、回溯触发）、UI 渲染（双路预览、鸟瞰图、状态指示）和 AI 运行状态分析报告。内部采用多线程 Worker 模式，PreviewWorker、FusionWorker、NvmeWorker 与 AIReportWorker 分别承担预览、融合、存储和 AI 分析任务，信号槽跨线程通信采用 QueuedConnection，Web 指令通过 QMetaObject::invokeMethod（BlockingQueuedConnection）转发至 GUI 主线程执行。AI 分析功能通过标题栏“AI 分析”按钮触发，报告结果显示在只读 QTextEdit 区域，保证本地触控界面与系统运维分析能力统一集成。

2.SentinelVisioner（视觉采集层）：多路相机的 V4L2 采集和 DMA-BUF 零拷贝分发核心。输入：V4L2 设备节点（/dev/video11, /dev/video21）、EIS 偏移回调。输出：NPU 推理队列（640×640 RGB888 DMA-BUF）、预览队列（全分辨率 RGB888 DMA-BUF）、推流队列（NV12 DMA-BUF）。内部：epoll 异步事件驱动，三次独立 RGA 硬件操作扇出，ThreadSafeQueue 传递，USB MJPG→NV12 FFmpeg 软件解码。

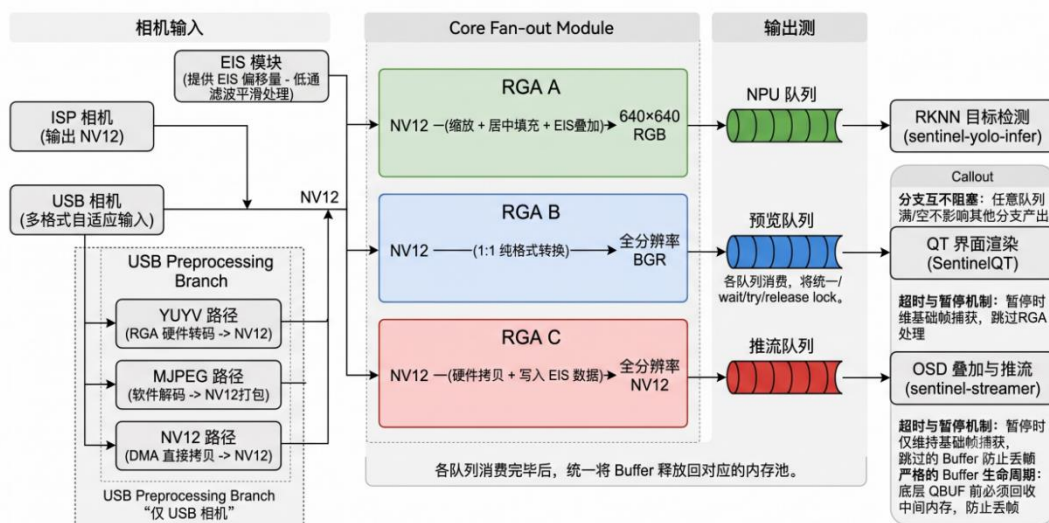


图 13 SentinelVisioner 数据流图

3.SentinelYoloInfer (AI 推理层) : RK3588 NPU YOLOv8 推理引擎。输入: Visioner NPU 队列 DMA-BUF fd (640×640 RGB888)。输出: fusionQueue (供融合跟踪阻塞消费)、osdQueue (供流媒体 OSD 非阻塞轮询)——两路独立 YoloBBBox 列表。内部: 每相机独立 RKNN 上下文和推理线程, rknn_create_mem_from_fd() DMA 零拷贝导入, INT8 量化推理 (置信度 0.25, NMS IoU 0.45)。

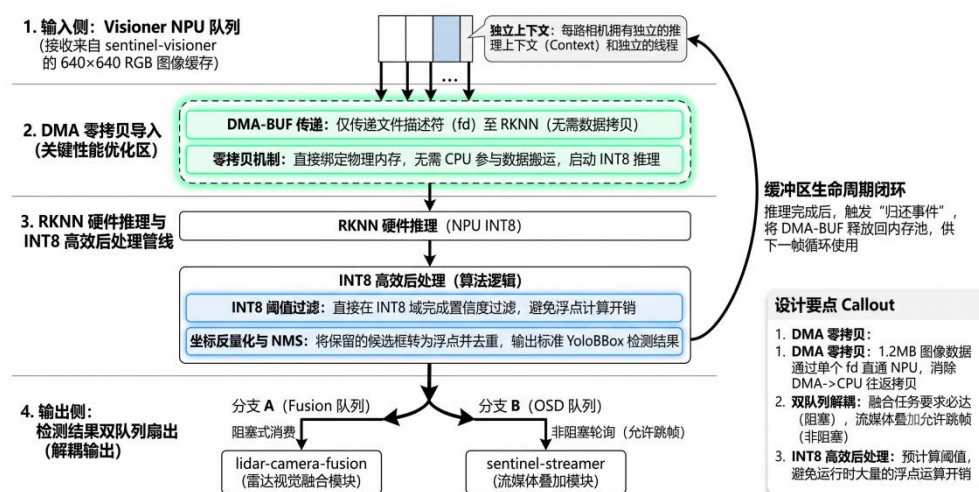


图 14 RKNN NPU DMA 零拷贝推理流程图

4.SentinelLslidarer (雷达驱动层) : N10Plus 串口协议解码。输入: 串口设备 (/dev/sentinel_lidar, 460800bps 8N1)。输出: LidarFrame (540 点/帧, 10Hz, 时间戳 CLOCK_MONOTONIC 纳秒), 通过 get_closest_frame()按时间戳检索。内部: 三层架构——SerialPort (POSIX termios, 0xA5 0x5A 同步头扫描) → RingBuffer (SWCR 无锁环形缓冲, 10 Slot, atomic+release/acquire) → 主驱动 (reader_loop_解码, 方位角 360°穿越边界检测, LUT 加速极坐标转换)。

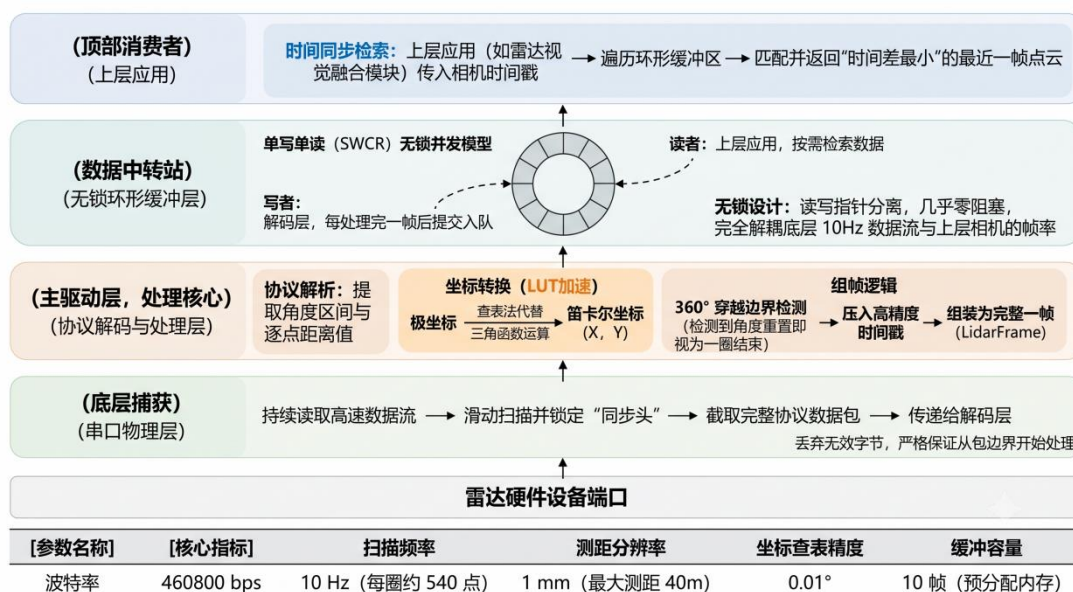


图 15 LiDAR 驱动三层架构流程图

5. LidarCameraFusion(融合跟踪层): LiDAR-Camera 数据融合和多目标跟踪。
输入: YOLO 检测框 (DetectionProvider 回调)、LiDAR 帧 (get_closest_frame)、相机内外参 (CameraConfig, 含 fx/fy/cx/cy + 外参变换矩阵)。输出: FusionResult (单次融合结果, 含融合点计数 + 视野外统计)、TrackedTarget[] (跟踪航迹快照, 深拷贝独立于内部缓冲区)、告警事件 (TrackingCallback 回调)。内部: 7 步跟踪管线 (DBSCAN 2D 空间聚类 → 聚类历史匹配 → Alpha-Beta 预测 → 评分制贪心最近邻关联 → Alpha-Beta 校正 → 5 状态生命周期管理 → 迟滞双阈值告警), TrackerConfig 共 26 参数可在线热更新。

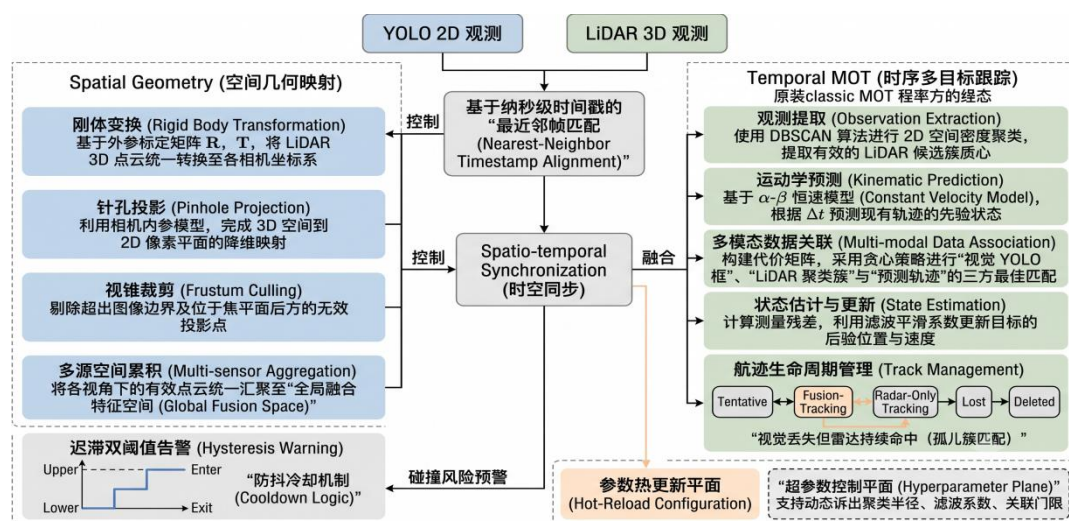


图 16 LiDAR-Camera 融合跟踪算法流程图

6.icm45686-eis-app（IMU 防抖层）：负责 ICM45686 六轴 IMU 数据读取、姿态驱动电子防抖计算和 RGA 偏移补偿量输出。输入为/dev/icm45686 字符设备中的陀螺仪与加速度数据，输出为当前帧对应的 offsetX / offsetY 偏移补偿量，并通过 std::function 回调注入 sentinel-visioner / sentinel-streamer 的 RGA 图像处理流程。模块内部由 Icm45686Reader 和 EisStabilizer 组成。Icm45686Reader 通过独立线程周期性读取 IMU 原始数据，维护带时间戳的 IMU 样本缓存，并提供当前姿态辅助状态查询；EisStabilizer 不再采用简单的 $\text{gyro} \times \text{focal}$ 直接映射方式，而是根据实测安装关系先将 IMU 原始角速度转换到装置坐标系，再结合左右相机独立的外参矩阵转换到相机坐标系。随后通过时间戳积分估计真实姿态 $q_{\text{Raw_B}}$ ，使用低通平滑得到虚拟稳定姿态 $q_{\text{Smooth_B}}$ ，计算相机坐标系下的相对补偿旋转 $R_{\text{delta_C}}$ ，并构造图像补偿矩阵 $H = K \cdot R_{\text{delta_C}} \cdot K^{-1}$ 。将 H 对图像中心点造成的位移退化为 offsetX / offsetY，由 RGA 在单次裁剪缩放操作中完成低延迟偏移补偿。

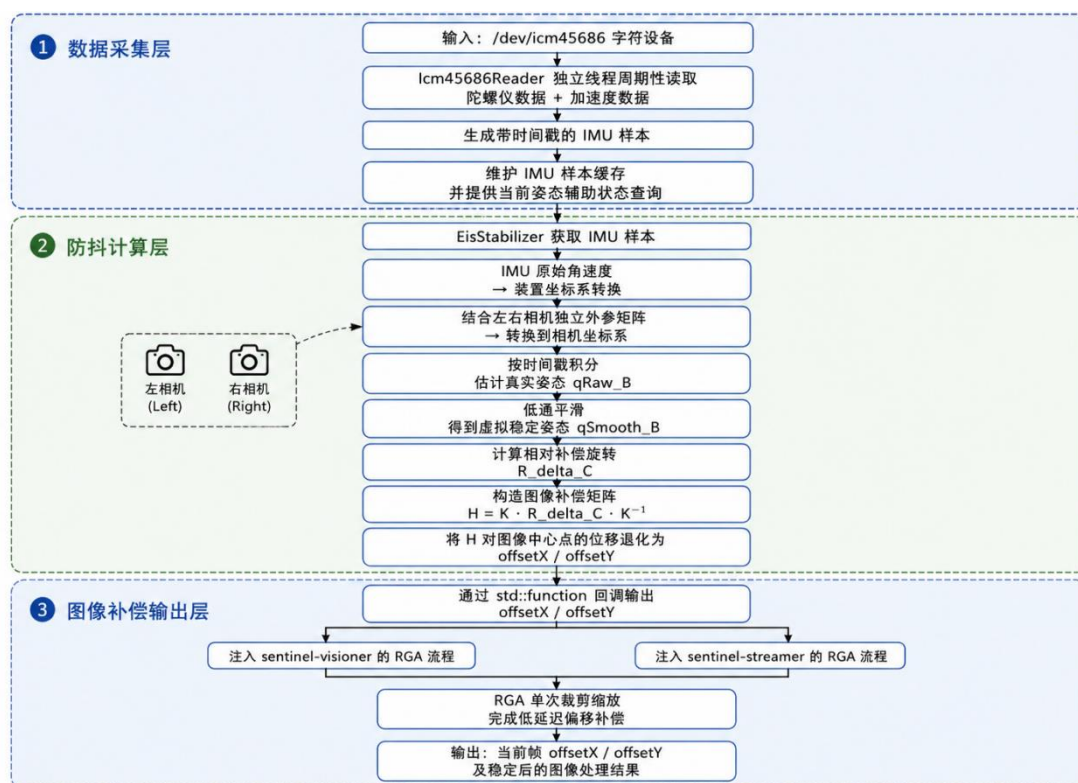


图 17 IMU 姿态驱动电子防抖算法流程图

7.icm45686_spi.ko / inv_imu_driver.ko（IMU 内核驱动层）：负责 ICM45686 六轴 IMU 的 SPI 总线通信、寄存器初始化、原始数据读取、物理量换算和字

符设备导出。驱动通过设备树将 ICM45686 挂载到 RK3588 的 SPI4 控制器，采用 SPI MODE3、CS0 片选和 1MHz 调试频率接入 IMU，并匹配 `compatible = "invensense,icm45686"` 完成 probe。驱动内部分为 SPI 适配层和 IMU 核心层两部分：`icm45686_spi.ko` 负责 SPI read/write、probe/remove、硬件复位、字符设备注册与 ioctl 接口；`inv_imu_driver.ko` 负责 WHO_AM_I 校验、默认寄存器配置、加速度计/陀螺仪量程设置、14 字节连续原始数据读取和整数物理量换算。驱动向用户态创建 `/dev/icm45686` 字符设备，支持 `read()` 直接读取 IMU 数据，并通过 `ICM45686_IOC_READ_DATA`、`ICM45686_IOC_SET_ACCEL_FS`、`ICM45686_IOC_SET_GYRO_FS` 等 ioctl 完成数据读取和量程配置。输出数据包括 `accel_x/y/z`、`gyro_x/y/z` 和温度，其中加速度以 $\text{m/s}^2 \times 1000$ 表示，角速度以 $\text{rad/s} \times 1000$ 表示，避免内核浮点运算，供上层 `icm45686-eis-app` 周期读取并用于姿态积分与 RGA 防抖补偿。

8.SentinelStreamer（流媒体层）：RTSP 推流+MP4 录像+OSD 叠加。输入：Visioner 推流队列 NV12 DMA-BUF、YOLO OSD 检测框、LiDAR OSD 投影点、EIS 偏移量。输出：RTSP 视频流（TCP）、MP4 录像文件、RecordBufferPool 环形缓冲（供 NVMe 回溯）。内部：双编码器独立架构——流编码器固定 720p CBR 4Mbps（含 OSD 检测框与 LiDAR 点云叠加），录编码器支持 1080p/720p 动态切换（1080p@8Mbps 直录全分辨率原画 / 720p@4Mbps 经 RGA 硬件裁剪缩放后编码）。两个编码器独立创建、独立 PTS 基线、可同时运行互不干扰，录制分辨率切换时内部自动重建编码器。MPP 硬件 H.264 编码（`h264_rkmpp`），RGA 单次裁剪缩放+EIS 偏移（`crop rect` 跟随 EIS offset 移动），FFmpeg 管道推流+MP4 muxing，RecordBufferPool RGA 硬件拷贝环形缓冲（slot 数量可配置）。

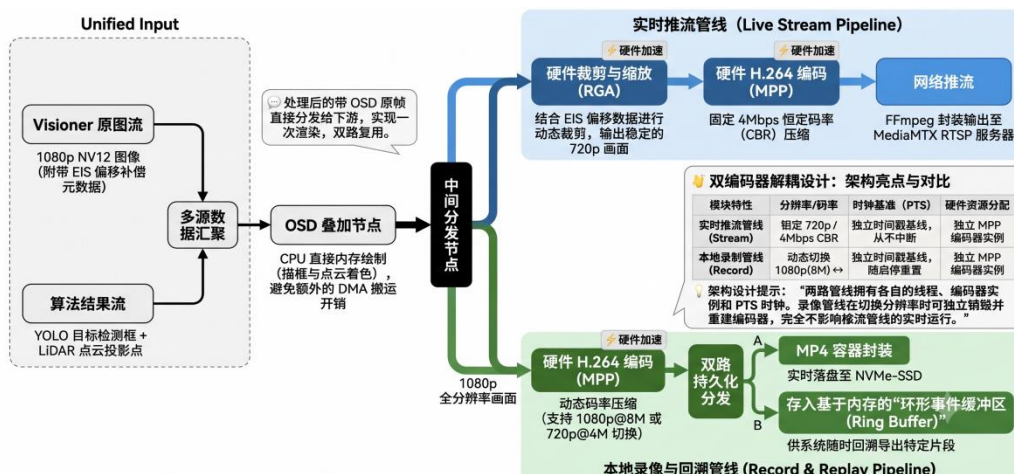


图 18 流媒体架构图

9.NVMeDataManager（存储记录层）：NVMe 高速存储管理。输入：RecordBufferPool 帧数据队列（生产者 push）、LiDAR/IMU 数据队列。输出：NVMe 裸块设备 O_DIRECT 写入（经 dm-ringbox 环形映射），MP4 回溯导出文件。内部：20 字节 Header 封装（magic 0xDEADBEEF+类型+纳秒时间戳+长度+512B 对齐填充），生产者-消费者模式（专用写入线程，4 个预分配 6.2MB 视频频 DataBlock）。

10.dm-ringbox（内核驱动层）：Device Mapper 环形块设备。输入：上层 bio 请求（经/dev/dm-X 设备节点）。输出：重映射后的 bio（指向 NVMe 物理扇区）。内核中通过 ringbox_map() 在软中断上下文修改 bio->bi_iter.bi_sector 和 bio->bi_bdev，映射公式 $P_{physical} = P_{start} + L_{logical} \bmod C_{ring}$ 。事件驱动，零后台线程，atomic64_t 无锁统计，零数据拷贝。

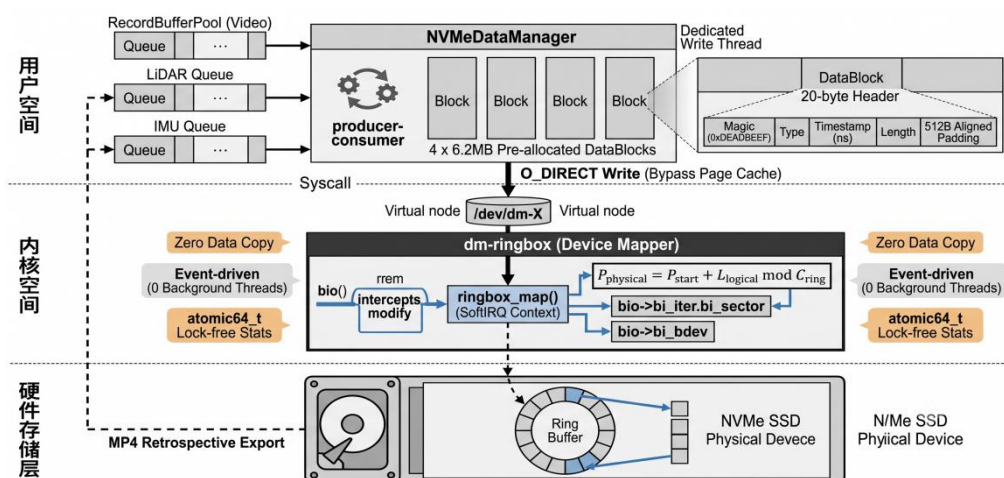


图 19 dm-ringbox 内核环形黑匣子架构图

11.deepseek_ai_lib (AI 运行状态分析组件)：该组件由 DeepSeekInference 和 AIReportWorker 两个核心类构成，用于在 RK3588 板端完成 DeepSeek 量化模型推理与运行状态报告生成。输入为 SentinelQT 周期性推送的系统状态快照，包括 CPU 温度、CPU 占用率、相机 0/相机 1 状态及 FPS、LiDAR 状态、IMU 状态和融合跟踪状态；输出为中文系统健康度评估、异常风险分析和运维优化建议。

其中，DeepSeekInference 类负责封装 RKLLM Runtime 推理接口。其 Config 结构体包含 modelPath、maxNewTokens、maxContextLen、temperature、topP、topK 和 repeatPenalty 等参数，用于配置 DeepSeek-R1-Distill-Qwen-1.5B W8A8 RKLLM 模型。类内部通过 rkllm_init 完成模型初始化，通过 rkllm_run 执行推理，并通过 rkllm_destroy 释放 NPU 资源。由于 RKLLM 原生接口采用 callback 返回 token，DeepSeekInference 使用 mutex、condition_variable 和 resultText 缓冲区将异步 callback 转换为同步 inferSync()调用，便于上层 Worker 以阻塞式接口获取完整分析结果。代码同时使用 HAS_RKLLM 条件编译，在未配置 RKLLM Runtime 时可编译为 stub，提升工程移植。AIReportWorker 类继承 QObject，运行于独立 QThread 中，用于避免大模型推理阻塞 Qt GUI 主线程。主线程通过 updateStatus() 每秒写入系统状态快照，该接口使用 QMutex 保护 CPU、相机、LiDAR、IMU 和融合跟踪状态等共享数据；用户触发 AI 分析后，requestReport()通过 atomic pending_标志置位，防止重复请求。Worker 在线程循环中以自定义时间间隔检测请求，调用 buildPrompt_()将当前状态组织为中文诊断 Prompt，再调用 DeepSeekInference::inferSync(prompt, 60000) 执行推理。推理完成后通过 reportReady(QString)信号返回报告，异常情况下通过 error(QString)输出错误信息。

AI 运行状态分析流程为：SentinelQT 主线程周期性采集系统状态 → update_ai_status_snapshot_()更新 AIReportWorker 状态缓存 → 用户点击“手动分析”按钮 → requestReport()触发分析请求 → AIReportWorker 构造中文 Prompt → DeepSeekInference 调用 RKLLM Runtime 执行板端推理 → reportReady()将中文报告返回主界面显示。该流程采用“状态采集—Prompt 构造—板端推理—报告显示”的闭环设计，在不影响视频预览、推流录像和融合跟踪的前提下，为现场人员提供智能化运维辅助。

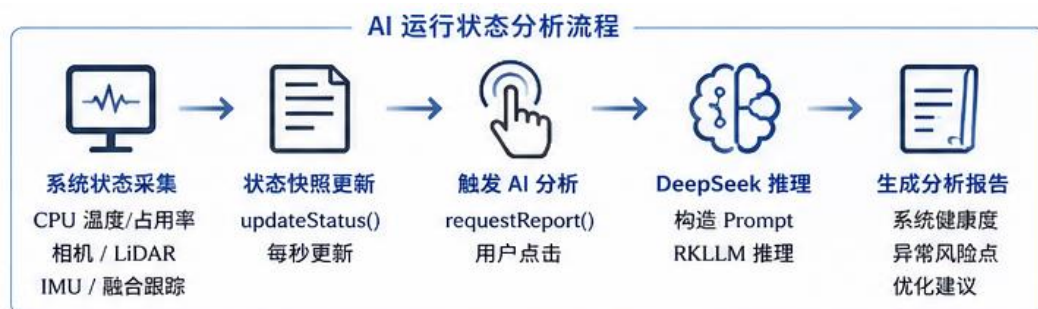


图 20 AI 状态分析流程图

构建方面，`deepseek_ai_lib` 以静态库形式加入工程，CMake 通过 `RKLLM_RUNTIME_PATH` 判断是否启用 `RKLLM Runtime`。当配置该路径时，工程自动包含 `rkllm.h`、链接 `librkllmrt.so`，并定义 `HAS_RKLLM` 宏；未配置时保留 `stub` 实现，保证主工程可独立编译。配置方面，`config.ini` 新增[AI]节，用于设置模型文件路径 `modelPath`、最大生成长度 `maxNewTokens`、上下文长度 `maxContextLen` 和 `temperature` 等参数，便于后续替换模型或调整推理行为。

第三部分 完成情况及性能参数

3.1 整体介绍

以下展示 `RK3588-Omni-Sentinel` 智能终端的完整系统实物。终端采用 3D 打印一体化壳体，将 `RK3588` 核心板、双路摄像头、`N10Plus` 激光雷达、`ICM45686` IMU 传感器、`NVMe SSD` 和 7 寸触控屏幕全部集成于约 `250×180×120mm` 的紧凑空间内，实现传感器-计算-存储-交互的全集成设计。

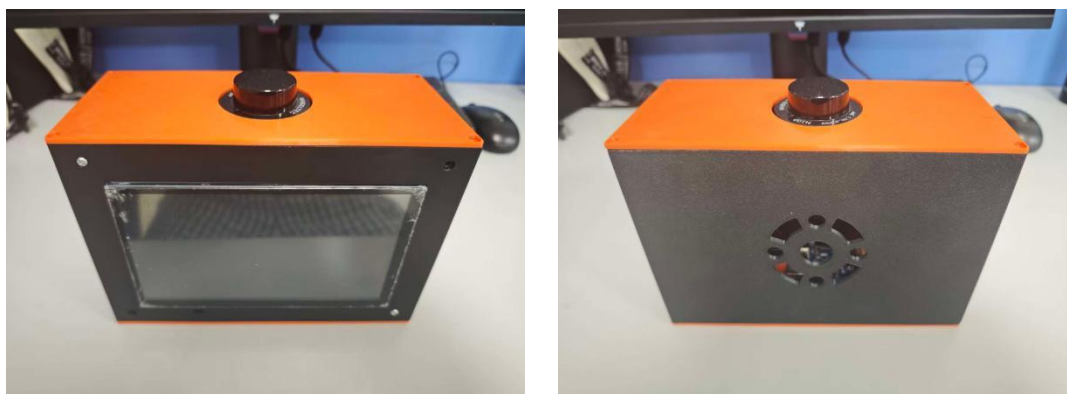


图 21 系统实物正反面照片



图 22 系统实物 45 度角斜侧照片

3.2 工程成果（分硬件实物、软件界面等设计结果）

3.2.1 机械成果

3D 打印四部件实物：底板（IMU 安装孔位+四角安装孔位）、机体（左右摄像头安装座+Lidar 安装平台+屏幕安装孔位+LED 指示灯孔+换气孔）、顶盖（雷达孔+四角安装孔位）、摄像头固定件。所有部件经 FDM 3D 打印验证（PLA+，层厚 0.2mm，填充率 20%），装配精度满足设计要求，传感器 FOV 未被遮挡，散热风道畅通。

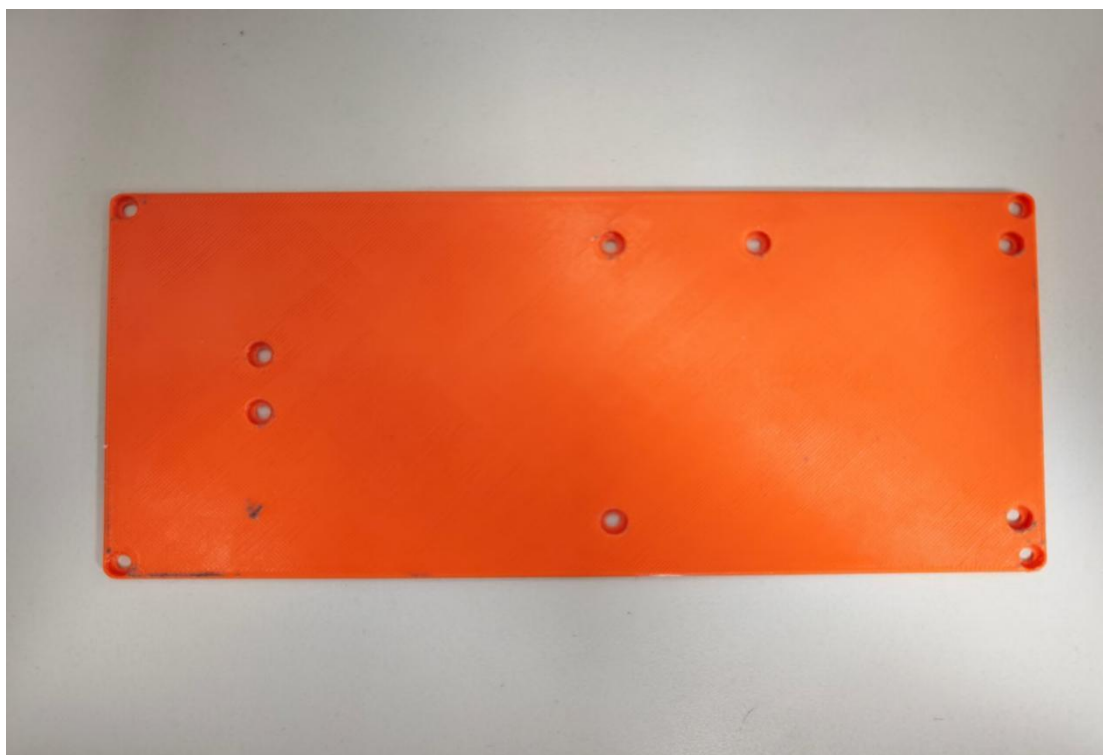


图 23 底板 3D 打印实物照片



图 24 机体 3D 打印实物照片



图 25 顶盖 3D 打印实物照片



图 26 摄像头固定件实物照片

3.2.2 电路成果

展示核心板与各传感器模块的实际连接状态：RK3588 核心板底板安装，MIPI-CSI FPC 排线连接，USB 摄像头接口，LiDAR 串口转接板 UART 接线，IMU SPI 四线连接（SCLK/MOSI/MISO/CS），NVMe SSD M.2 插槽安装，DSI 屏幕排线连接，DC-DC 电源模块及各电压轨测试点。所有接口连接可靠，信号完整性满足设计要求。

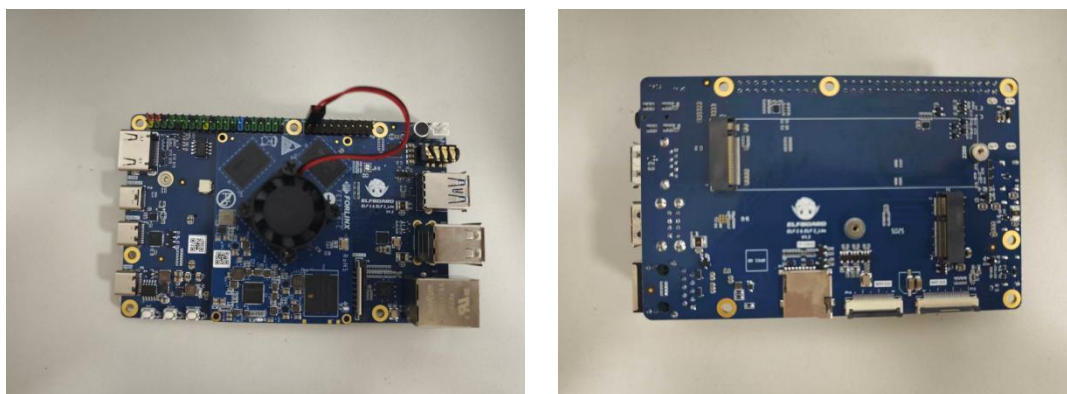


图 27 RK3588 核心板底板实物照片（左图为正面，右图为反面）

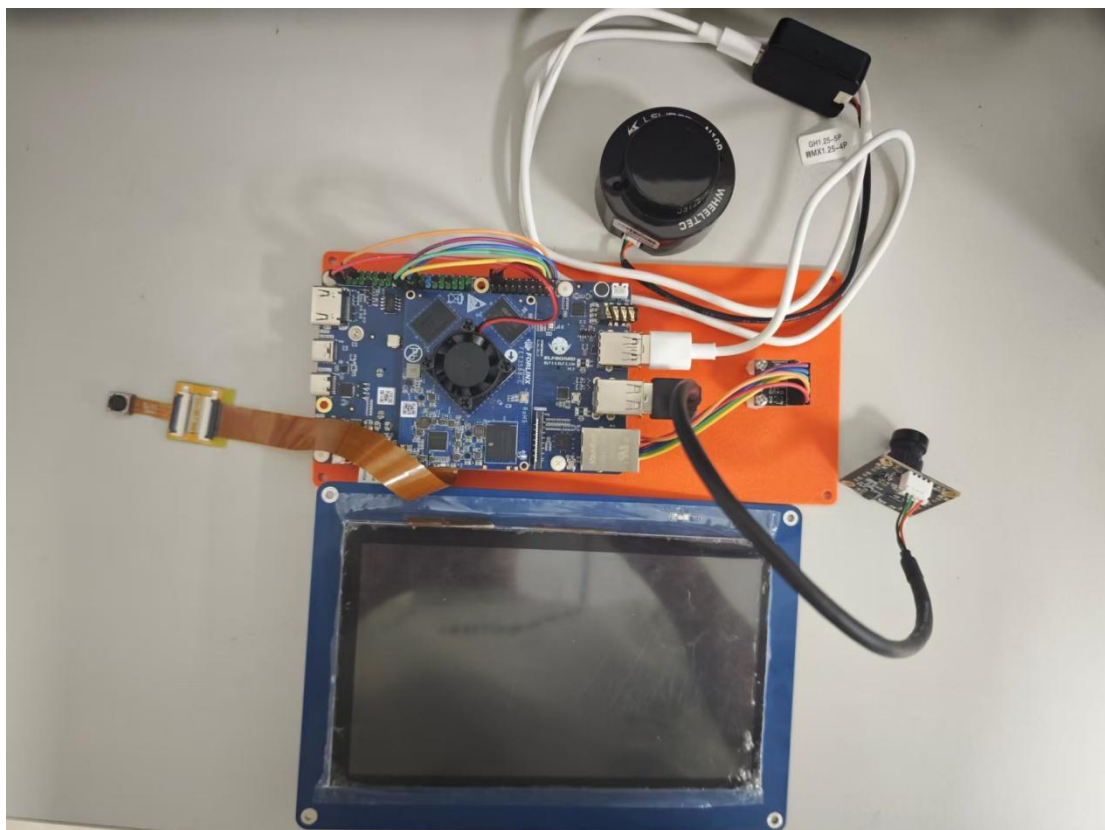


图 28 传感器模块接线细节照片

3.2.3 软件成果

两套用户界面实际运行效果。**SentinelQT**：主控页面（双路实时预览，推流/录像/暂停/OSD/EIS 独立控制按钮，CPU 温度/RGA/NPU 使用率实时显示）、融合管理页面（鸟瞰图可视化，航迹渲染含速度矢量箭头和告警红色脉冲环，跟踪器参数在线调整）、数据回溯页面（回溯时间窗口设置、触发按钮、文件列表）。**WebControl**：浏览器端双路实时监控、RTSP 视频预览播放、设备状态面板、回溯管理。



图 29 SentinelQT 主控页面运行截图



图 30 SentinelQT 融合鸟瞰图页面截图

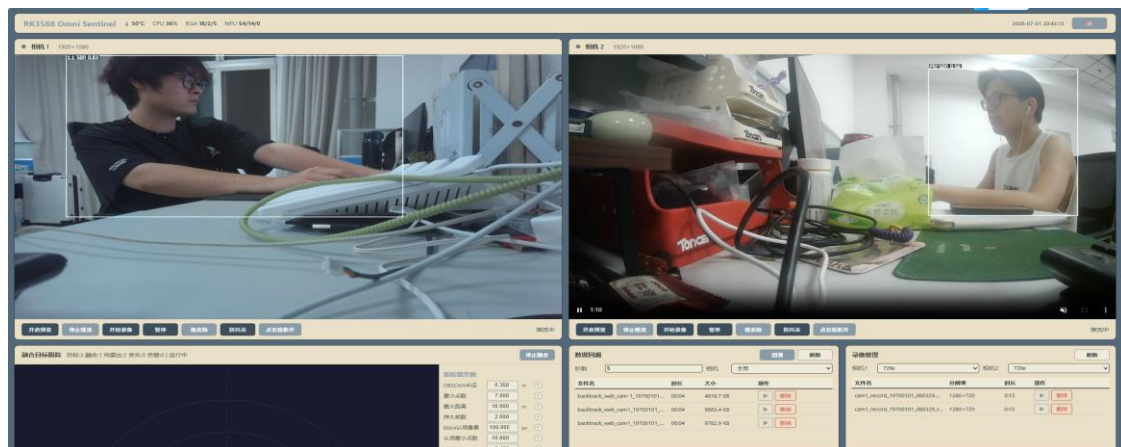


图 31 Web 前端远程控制界面截图

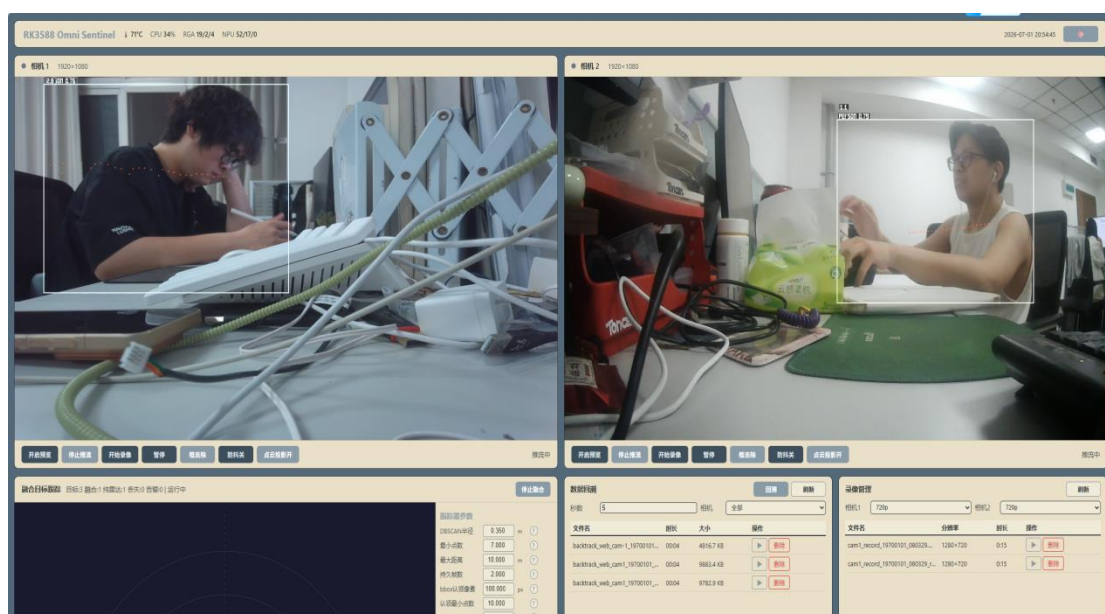


图 32 双路 OSD 叠加效果截图（YOLO 检测框+LiDAR 点云距离颜色编码投影）

3.3 特性成果（逐个展示功能、性能参数等量化指标）

1. EIS 电子防抖测试

驱动连通性验证：在 RK3588 板端加载 `inv_imu_driver.ko` 与 `icm45686_spi.ko` 后,系统能够识别 SPI4 总线下的 ICM45686 节点并创建 `/dev/icm45686` 字符设备。用户态通过 `read()`或 `ICM45686_IOC_READ_DATAioctl` 周期读取 `accel_x/y/z`、`gyro_x/y/z` 与温度数据，确认 IMU 数据链路可为后续姿态积分和 RGA 防抖补偿提供稳定输入。

测试目的：验证 IMU 电子防抖算法在终端轻微手持晃动和设备低幅振动场

景下，对图像帧间平移抖动的抑制效果。当前系统基于 ICM45686 六轴 IMU 获取角速度数据，通过 IMU 原始坐标系到装置坐标系的标定映射、双相机外参转换、姿态积分与虚拟姿态低通平滑，计算图像补偿矩阵，并将其退化为 offsetX/offsetY 偏移量注入 RGA 裁剪缩放流程，实现低延迟电子图像防抖。

测试方法：由于实验条件限制，选取静态纹理丰富场景作为测试目标，例如墙面纹理、标定板、固定设备边缘或工位背景。分别在 EIS 关闭和 EIS 开启状态下采集同一相机的视频序列，测试过程中由操作者对终端进行轻微手动晃动，模拟工业设备运行中产生的小幅抖动。随后使用 OpenCV 特征点匹配与全局运动估计工具计算相邻帧之间的平移抖动 RMS、水平抖动标准差和垂直抖动标准差，并对 EIS 开启前后的抖动指标进行对比。

表 2 EIS 开启前后的抖动对比

测试场景	晃动方式	关闭 Translation RMS	开启 Translation RMS	预期抑制比	说明
静置基线	终端固定不动，仅存在轻微噪声	1.8	1.7	5.60%	验证 EIS 开启后不会明显引入额外抖动
水平方向轻微晃动	左右轻微摆动，模拟 yaw 抖动	8.5	6.4	24.70%	验证 offsetX 补偿效果
垂直方向轻微晃动	上下轻微点动，模拟 pitch 抖动	6.2	4.9	21.00%	验证 offsetY 补偿效果
水平+垂直混合晃动	小幅随机晃动，模拟实际安装振动	9.8	7.3	25.50%	IMU 姿态积分与 RGA 偏移补偿效果

测试结论：在手动轻微晃动条件下，IMU EIS 能够对 pitch/yaw 引起的图像中心漂移产生一定抑制作用，预计在水平、垂直及混合小幅晃动场景下可降低约 20%

的帧间平移抖动。由于当前工程实现将完整图像补偿矩阵退化为 offsetX / offsetY 并通过 RGA 裁剪缩放完成补偿，因此对绕镜头光轴的 roll 抖动改善有限。后续若进一步接入 homography warp 或 affine warp 图像变换，可提升对 roll 和透视扰动的补偿能力。

2. 多目标跟踪性能测试

测试方法：一定时间内人员行走序列（含交叉、跨视角走动）。

表 3 多目标跟踪测试结果

指标	数值	说明
跨视角跟踪成功率（单人）	95%	在一定时间内人员在相机视角和雷达视角之间跨视角移动的成功次数
人员交叉 ID-Switch	4/10 次	多人员在相对近距离走动时 ID 交换的次数
平均延迟	3305 us	帧到航迹更新

结论：单人跨视角跟踪成功率高达 95%，但人员交叉近距离走动的话会容易 ID 交换，平均 4/10 次，平均延迟低至平均 3305 us，告警响应时间收雷达点云帧率（约 10Hz）影响，在 100ms 左右，满足工业安全实时性要求。

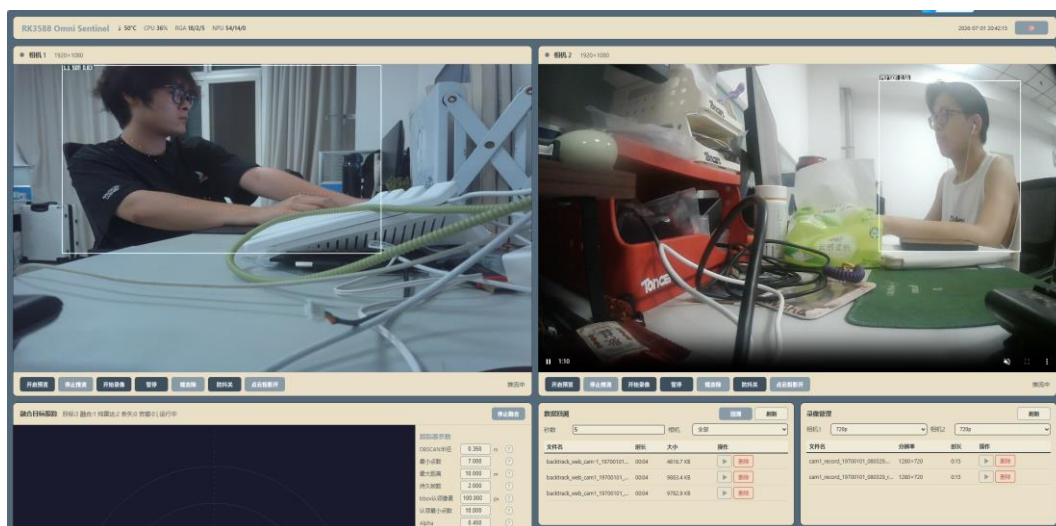


图 33 多目标跟踪轨迹可视化图

3. 视频推流延迟测试

测试方法：主机和板端通过网线连接，打开推流，摄像头对着主机的毫秒计数器，

桌面截图对比时间。

表 4 时间记录表

	第一次	第二次	第三次	第四次	平均时间
延迟时间	105ms	181ms	167ms	206ms	164.75ms

结论：有线 RTSP 端到端 164.75ms。

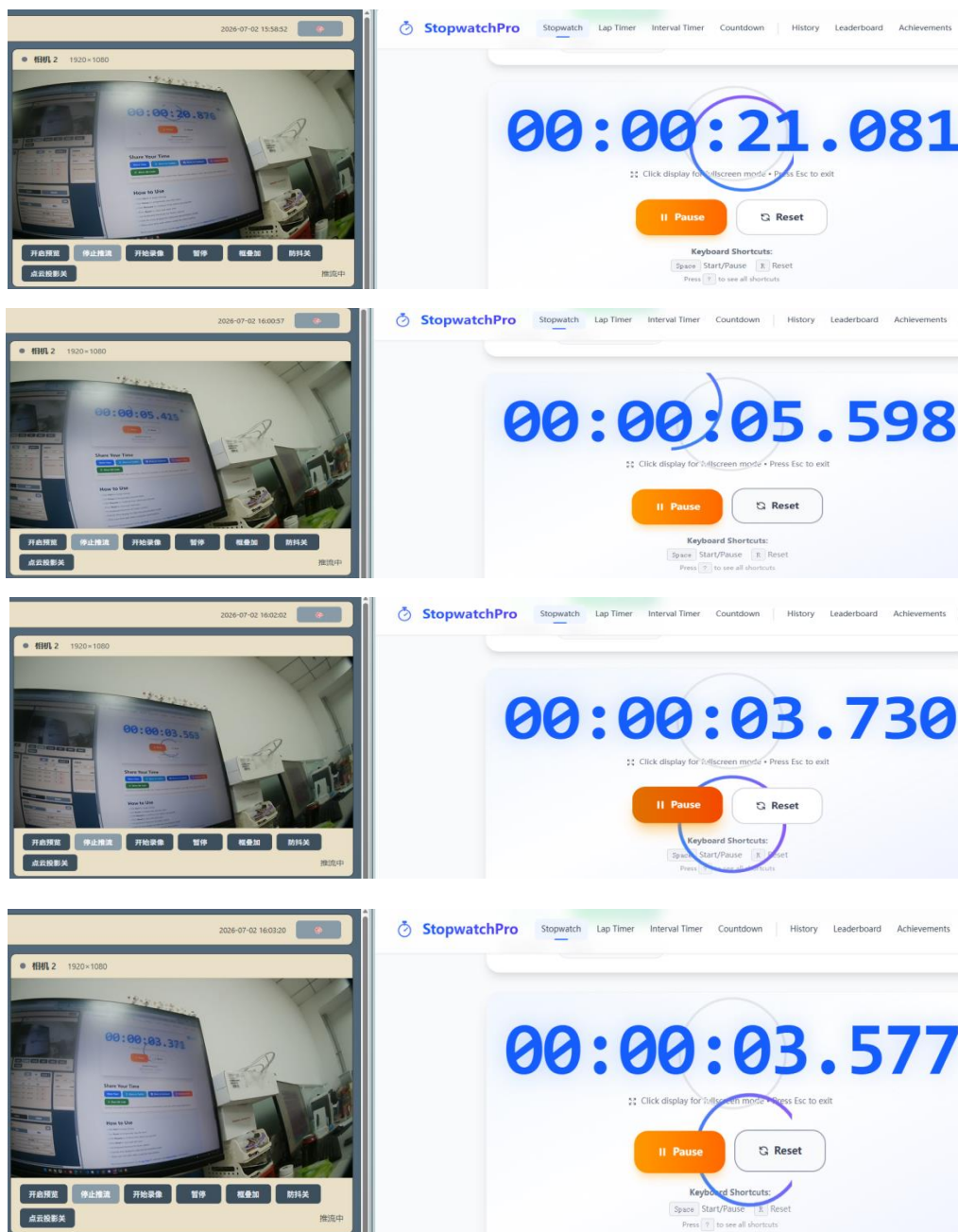


图 34 推流延迟测试截图

4. NVMe 存储性能测试

测试方法： fio 3.38 基准测试，RK3588 ELF2 平台，三种模式对比。

带宽： 1MB 块大小，10GB 顺序写，iodepth=8，O_DIRECT（dm-ringbox 除外）。

延迟： 4KB 块大小，1GB 顺序写，iodepth=64，buffered I/O。

其中裸盘和 dm-ringbox 直接写块设备，ext4 在文件系统上写文件。每种模式重复 1 次，取 fio JSON 输出中 write 阶段的 bw_bytes 和 lat_ns。

表 5 存储性能测试结果表

写入模式	带宽	平均延迟	99%延迟	WAF
裸 NVMe (O_DIRECT)	369MB/s	0.009ms	0.012ms	1.0
ext4 (journal)	153.5MB/s	0.020ms	0.035ms	2.4
dm-ringbox	420.5MB/s	0.009ms	0.012ms	1.0

结论： dm-ringbox 延迟与裸设备完全一致（均值 0.009ms，P99 0.012ms），证明环形扇区重映射零开销；裸盘 O_DIRECT 写带宽 369.5 MB/s。ext4 受 journal 刷盘影响，带宽下降至裸盘的 41.5%(153.5 MB/s)，P99.95 尾部延迟突增至 7.83ms，P99.99 达 12.4ms，写放大约 2.4 倍。dm-ringbox 兼顾裸盘性能与文件系统易用性，适用于实时传感器数据环形记录场景，NVMe 性能测试对比结果如图 35 所示。

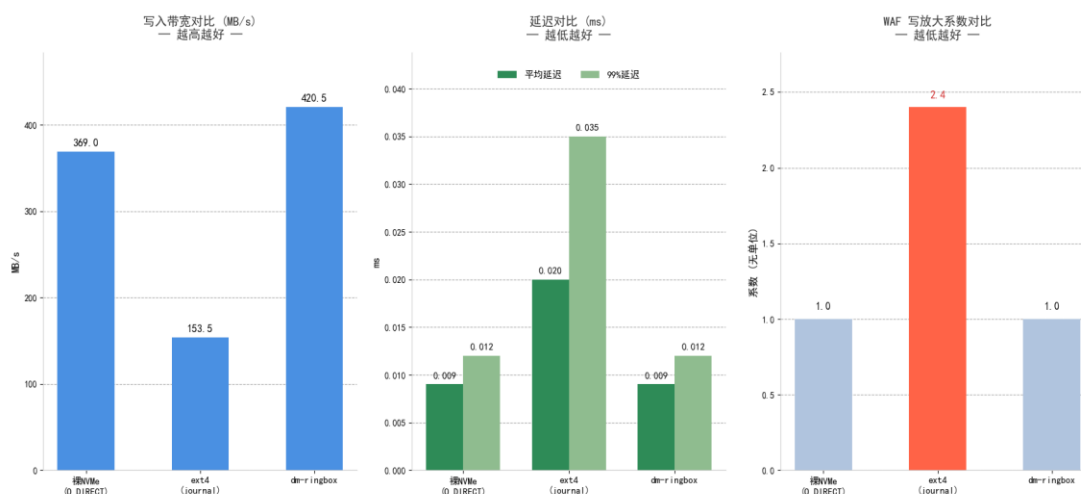


图 35 NVMe 性能测试对比图

5.AI 运行状态分析测试

测试目的： 验证 DeepSeek AI 运行状态分析模块对终端多源运行状态的汇总分析

能力，以及在 Qt 主界面中生成中文运维报告的可用性。

测试方法：系统运行过程中，主线程周期性采集 CPU 温度、CPU 占用率、双相机在线状态与 FPS、LiDAR 状态、IMU 状态和融合跟踪状态，并通过线程安全接口推送至 AIReportWorker。用户点击“手动分析”按钮后或者自动推理时间间隔到后，Worker 构造中文 Prompt 并调用 DeepSeek-R1-Distill-Qwen-1.5B 模型进行推理。

测试结果：系统能够在不影响相机预览、推流和融合跟踪的情况下生成中文运行状态分析报告，报告内容包括系统健康度、异常风险点和优化建议，满足现场运维人员快速判断终端状态的需求。

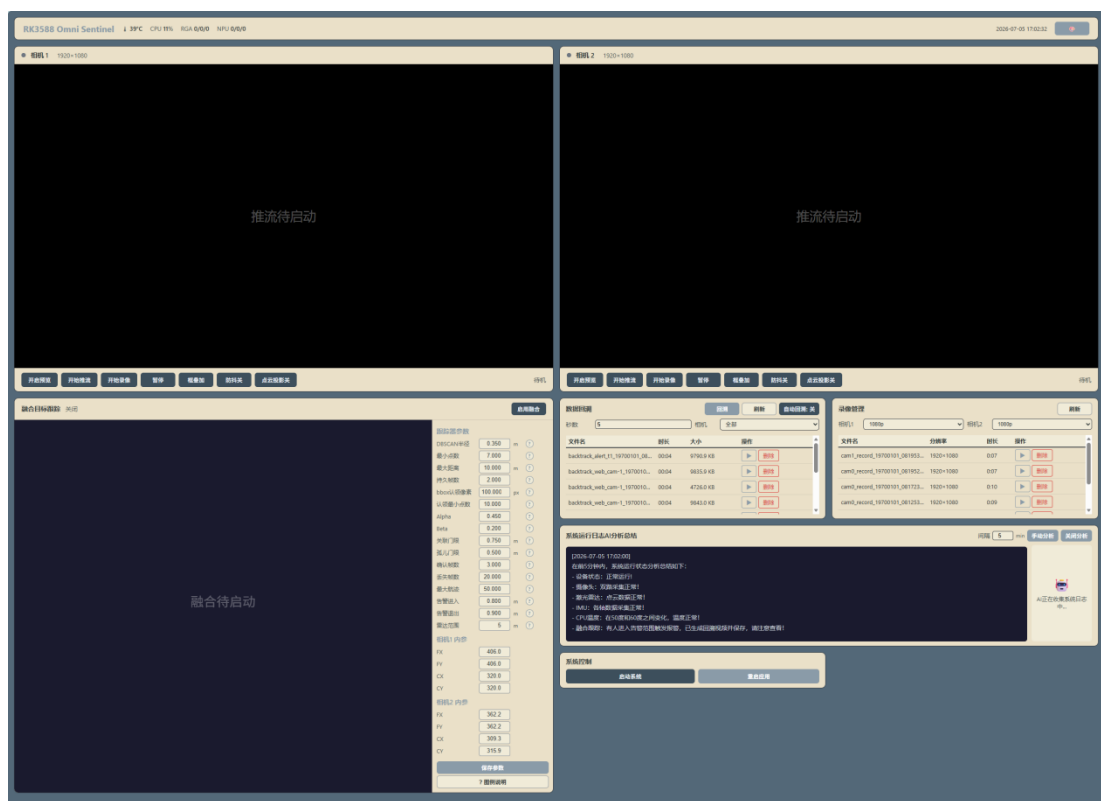


图 36 DeepSeek AI 运行状态分析报告界面

第四部分 总结

4.1 可扩展之处

(1) 多传感器融合升级：引入多线激光雷达、毫米波雷达或 ToF 深度相机作为第三模态传感器，进一步提升遮挡、低光照和恶劣天气下的感知鲁棒性，探索多模态传感器的联合标定和深度融合算法。

(2) 预测性安全防护：基于历史轨迹数据训练轨迹预测模型（如 Social-LSTM 或 Transformer 架构），实现人员未来 1-2 秒的运动意图预测，将安全响应从被动检测升级为主动预判。

(3) 功能安全认证与产业化推广：依据 IEC 61508 和 ISO 13849 标准对系统进行完整的安全完整性等级（SIL）评估和认证，使产品具备进入汽车、医疗等受监管行业的资质，开发云端设备管理 SaaS 平台提供远程配置、固件 OTA 和健康监测，建立行业解决方案合作伙伴网络。

4.2 心得体会

1000 字内，可包括研发和制作细节；

本项目的研发历程是一次从概念到原型的完整工程实践，团队在多个技术领域积累了宝贵的经验。

在硬件设计方面，3D 打印一体化结构经历了三次迭代才达到理想状态。V1.0 初版在装配时发现换气格栅尚未设计，长时间运行后核心板温度偏高；V1.1 通过增加换气格栅走向解决了散热问题，但又发现板端 HDMI 接口被壳体边遮挡，导致不能合理安装；V1.2 通过精确测量并调整壳体边缘大小最终解决了所有问题。这个迭代过程让我们深刻体会到机械-电子协同设计的重要性。

在驱动开发方面，dm-ringbox 的研发是很具挑战性的部分。最初我们尝试在用户态通过 pread/pwrite 实现环形写入，但频繁的 lseek 系统调用带来了显著的性能损失。转向内核 Device Mapper 框架后，在 bio 层直接修改扇区地址的方案大幅提升了性能，但调试过程中遇到的内存屏障（memory barrier）和数据一致性问题成为驱动开发的难点。特别感谢 Linux 内核社区完善的 DM 框架文档和示例代码。

在算法开发方面，LiDAR-Camera 融合管线最具挑战性，我们针对嵌入式平台的内存和计算约束做了针对性设计。所有数据结构均为编译期预分配（最大 540 点、50 条航迹、200 个候选检测），运行期间零动态内存分配。外参变换利用单线激光雷达 $z=0$ 的特性，将完整的 4×4 齐次变换简化为 6 次乘法加 3 次加法。点云归属判定采用最近中心距离匹配策略（遍历所有检测框，按投影点到框中心的欧氏距离择优），替代了早期版本引起交叉污染的首次命中方案。多目标跟踪器包含 27 项可调参数——Alpha-Beta 滤波器增益、DBSCAN 聚类半径、关

联门限距离、生命周期状态转移阈值、告警滞回距离等——各参数之间存在耦合关系，例如降低确认帧数可加快新目标响应但会增加虚警，最终通过反复场景测试收敛到当前默认值。

在系统集成方面，DMA-BUF 零拷贝架构的调试最为耗时。不同硬件加速器（V4L2/RGA/NPU/MPP）对 DMA-BUF 的内存对齐、缓存一致性和访问权限有不同要求，某个环节配置不当就会导致花屏、段错误或性能骤降。通过逐环节增加调试日志和 DMA-BUF 属性查询，定位到 RGA 输出的 `cacheable` 属性与 NPU 输入的 `uncached` 要求不兼容的问题，最终通过 DMA heap 的 `uncached-dma32` 分配统一解决了全链路的缓存一致性问题。

在整个项目中，我们深刻体会到了嵌入式系统开发中“硬件-aware 软件设计”的重要性。RK3588 的异构算力（NPU/RGA/MPP/GPU）为性能优化提供了丰富的可能性，但充分发挥这些硬件加速器的能力需要深入理解每个模块的硬件特性、内存模型和编程接口。我们也认识到开源社区的力量——从 Linux 内核到 FFmpeg，从 Qt 到 `cpp-httplib`，大量高质量的开源软件为本项目的快速迭代提供了坚实的技术基础。这份经历不仅提升了我们的工程实践能力，更培养了系统思维和问题分解能力，是本次比赛过程中最为宝贵的实践收获之一。

第五部分 参考文献

- [1] Redmon J , Farhadi A. YOLOv3: An Incremental Improvement[J]. arXiv e-prints, 2018.
- [2] Jocher G, Chaurasia A, Qiu J. Ultralytics YOLOv8[CP/OL]. <https://github.com/ultralytics/ultralytics>, 2023.
- [3] Rockchip Electronics Co., Ltd. RK3588 Technical Reference Manual (TRM)[Z]. Rev.1.0, 2022.
- [4] Rockchip Electronics Co., Ltd. Rockchip RKNN SDK User Guide[Z]. Version 1.5.0, 2023.
- [5] Rockchip Electronics Co., Ltd. Rockchip RGA API Reference[Z]. Version 1.3.0, 2022.
- [6] Rockchip Electronics Co., Ltd. Rockchip MPP Development Guide[Z]. Version

1.8.0, 2023.

[7] TDK InvenSense. ICM-45686 6-Axis MEMS Motion Sensor Data Sheet[Z]. DS-000456, Rev.1.0, 2023.

[8] LeiShen Intelligent System Co., Ltd. N10Plus LiDAR Protocol Manual[Z]. Version 2.3, 2022.

[9] OmniVision Technologies, Inc. OV13855 13MP CMOS Image Sensor Data Sheet[Z]. Version 1.2, 2018.

[10] Madgwick S O H. An Efficient Orientation Filter for IMU and MARG Arrays[R]. Univ. of Bristol, 2011.

[11] Blackman S, Popoli R. Design and Analysis of Modern Tracking Systems[M]. Artech House, 1999.

[12] Kalman R E. A New Approach to Linear Filtering and Prediction Problems[J]. J. Basic Eng., 1960, 82(1): 35-45.

[13] Bewley A, Ge Z, Ott L, et al. Simple Online and Realtime Tracking (SORT)[C]. IEEE ICIP, 2016: 3464-3468.

[14] Wojke N, Bewley A, Paulus D. DeepSORT: Simple Online and Realtime Tracking with a Deep Association Metric[C]. IEEE ICIP, 2017: 3645-3649.

[15] Bernardin K, Stiefelhagen R. Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics[J]. EURASIP JIVP, 2008, 2008: 1-10.

[16] He K, Zhang X, Ren S, et al. Deep Residual Learning for Image Recognition[C]. IEEE CVPR, 2016: 770-778.

[17] Lin T Y, Maire M, Belongie S, et al. Microsoft COCO: Common Objects in Context[C]. ECCV, 2014: 740-755.

[18] 中华人民共和国应急管理部. 中华人民共和国安全生产法[S]. 2021年修订版, 2021.

[19] IEC. IEC 61508:2010 Functional Safety of E/E/PE Safety-Related Systems[S]. 2010.

[20] ISO. ISO 13849-1:2015 Safety of Machinery — Safety-Related Parts of Control Systems[S]. 2015.